CS 4850-02 • Spring 2026 • January 25, 2026

Professor Sharon Perry

PROJECT **SP-5Red**

Grocery List App

Final Report

276 Total Man Hours	**3874 (3613 Dart and 261 SQL)**	**7** Project Components

Joshua Dupati
Developer (Team Lead)

Ash Arul
Documentation / Testing

Jacob Brockel
Developer / Documentation

Website: https://happyshoppercs4850.github.io/HappyShoperWebsite/index.html
GitHub: https://github.com/HappyShopperCS4850/HappyShoperApp

Table of Contents

1. Introduction

1.1 Project Overview

This project focuses on the design and development of a cross platform grocery list application. The application will support user accounts and shared grocery lists. Multiple users will be able to view and update the same list in real time. All changes will sync across devices to keep data consistent.

The system will include user authentication and a centralized database for storing user and list data. The project scope includes requirements analysis, database design, interface mockups, application development, testing and deployment of a working prototype. The project follows standard software development life cycle practices and emphasizes collaboration, version control and testing.

1.2 Platform

Grocery List app development using Flutter and Dart. Flutter is an open-source UI SDK that allows for development of both iOS and Android apps. It is a Google product that works with the Dart programming language which is easy to learn and provides natively compiled, optimized applications.

1.3 Collaboration Tools

Communication	Teams, iOS Group Chat, Google Workspace
Collaboration	GitHub, Google Workspace, Slack

1.4 Statement of Participation

By submitting this assignment, I acknowledge that I will participate in all meetings, communications, deliverables and other tasks necessary to complete the project. If I do not participate, I understand that Professor Perry will meet with me to remedy the situation.

1.5 Definitions and Acronyms

SRS	Software Requirements Specification
SDD	Software Design Document
CRUD	Create, Read, Update, Delete
UI	User Interface
API	Application Programming Interface

2. Requirements

2.1 Project Scope

The scope of this project includes requirements analysis, system design, interface mockups, application development, testing, and deployment of a working prototype. The application will support user authentication, shared grocery lists, real time synchronization, and basic list management functionality.

2.2 Functional Requirements

2.2.1 User Account Management

- The system shall allow users to create an account using an email and password
- The system shall allow users to log in using valid credentials
- The system shall prevent unauthorized access to user data

2.2.2 Authentication

- The system shall authenticate users before granting access to grocery lists
- The system shall maintain a logged in user until logout

2.2.3 Grocery List Management

- The system shall allow users to create a new grocery list
- The system shall allow users to add items to a grocery list
- The system shall allow users to update grocery list items
- The system shall allow users to delete grocery list items

2.2.4 Shared Lists

- The system shall allow users to invite other users to a grocery list
- The system shall allow multiple users to view and edit the same grocery list
- The system shall synchronize grocery list updates in real time across all users

2.2.5 Navigation

- The system shall display a home screen listing all grocery lists associated with the user
- The system shall allow users to navigate between lists and application screens

2.3 Non-Functional Requirements

2.3.1 Security

- The system shall securely store user authentication credentials
- The system shall restrict access to grocery lists to authorized users only

2.3.2 Capacity

- The system shall support multiple users accessing the same grocery list simultaneously

2.3.3 Usability

- The application shall provide a simple and intuitive UI
- The application shall be usable by non-technical users

2.3.4 Performance

- Grocery list updates shall be reflected to all users within a reasonable time frame

2.4 External Interface Requirements

2.4.1 User Interface

- The application shall provide mobile-friendly screens optimized for touch interaction

2.4.2 Hardware Interface

- The application shall run on standard iOS and Android mobile devices

2.4.3 Software Interface

- The application shall interface with a backend database for data storage and synchronization

2.4.4 Communication Interface

- The application shall use network communications to support real time data

2.5 Constraints

2.5.1 Environment

- The application will be developed using Flutter / Dart
- The application must run on both iOS and Android platforms

2.5.2 System

- The application will rely on a centralized backend database to store user data and grocery lists
- Real time synchronization depends on network availability
- Must support both iOS and Android
- Time constraint is confined to the semester
- List editing is confined to authorized members
- Storage limitations of Supabase backend

3. Analysis / Design

3.1 Use Cases

- User creates an account and logs in
- User creates a grocery list
- User shares a grocery list with another user
- Multiple users update a shared grocery list simultaneously

3.2 Data Flow

Users' actions in the application are sent to the backend database, which processes and stores the updates. Updated data is then synchronized and displayed across all connected user devices in real time.

3.3 Design Considerations

3.3.1 Assumptions and Dependencies

- Users have an iOS or Android device
- Internet connectivity is available
- User authentication via email and password
- Users trust each other for list collaboration
- Flutter, Dart, PostgreSQL Database

3.4 Development Methods

The software design follows the Agile method of development. The process is iterative in nature and breaks the development process into small manageable segments for continuous delivery. Self-organization, collaboration, and flexibility are the focal points of our team's development strategy.

3.5 Architectural Strategies

3.5.1 Cross-Platform Framework

Cross platform framework will be developed through the use of Flutter and Dart from a single codebase. React Native was considered but the all-in-one self-contained toolkit of the Flutter SDK was chosen.

3.5.2 Backend Architecture

The open source, back-end-as-a-service platform Supabase was used for backend development. Supabase provides authentication, storage, and real time data synchronization. Connection to Supabase is done through a RESTful API foundation. Supabase streamlined the application and data layer of our 3-tier client server architecture.

3.5.3 Database

PostgreSQL with its relational database format was chosen to store users' accounts and list information. The relational model provides data integrity and clearly identifies connections between users, groups, and list information.

3.5.4 Real-Time Synchronization

List updates between multiple users in a shared list is as close to real time as possible. Supabase monitors changes to the PostgreSQL write ahead log. Once changes are detected updates are broadcasted to subscribed users over websockets.

3.5.5 Authentication

Authentication is ensured using JSON Web Tokens (JWTs). Access tokens confirm user ID in each API request and refresh tokens prevent unwanted signouts. Credentials are located in the system auth table with hashing for further encryption. Row level security ensures users can only read, write, update, or delete appropriate rows.

3.6 System Architecture

The system is organized using a three-tier client-server architecture. The presentation tier and UI is handled by Flutter built using the Dart language. The application tier is handled by Supabase. API connections and authentication are provided and configured within Supabase. The data tier is handled by our PostgreSQL database stored with Supabase.

4. Development

4.1 Detailed System Design

Most components described in the System Architecture section will require a more detailed discussion. Other lower-level components and subcomponents may need to be described as well. Each subsection of this section will refer to or contain a detailed description of a system software component.

4.1.1 Classification

The SP-5Red Grocery List App follows a client-server architecture. The system consists of a mobile front end application and a centralized backend service. The front end handles user interaction while the backend manages data storage, authentication and synchronization.

The application will be developed using Flutter to support both iOS and Android platforms. A backend service will store user accounts, grocery lists and shared list data. Real time updates will be handled through network communication between the client and backend.

4.1.2 Definition

The system is divided into three main components:

- User Interface Component — allows users to log in, create grocery lists, edit items and share lists with others. This component focuses on usability and clear navigation.
- Application Logic Component — handles validation, list operations and communication with the backend. It processes user actions before sending or receiving data.
- Database Component — stores user credentials, grocery lists and shared list relationships. It ensures data persistence and supports real time synchronization across users.

4.1.3 Constraints

The application depends on network availability for real time synchronization. Users must have an active internet connection to view and update shared grocery lists. Offline access will be limited and updates may not sync until connectivity is restored.

The system relies on a centralized backend database. Storage limits and backend service availability may affect performance. Authentication is required before any list data can be accessed. Only authorized users may view or modify shared lists.

The application is constrained to mobile platforms supported by Flutter. Device performance and operating system limitations may impact responsiveness.

4.1.4 Resources

The application uses device resources such as memory, storage and network connectivity. A backend database is used to store user accounts, grocery lists and shared list relationships. A backend authentication service is required to manage user access.

Multiple users may access the same grocery list at the same time. This can create race conditions if updates occur simultaneously. These issues will be handled by backend synchronization logic and controlled write operations to ensure data consistency. Deadlock situations are unlikely due to the stateless nature of client requests.

4.1.5 Interface / Exports

The application provides services for user authentication, grocery list creation, item management and list sharing. These services allow users to create, edit, delete and view grocery lists through the mobile interface.

The system exposes interfaces for retrieving list data, updating items and synchronizing changes with the backend. Each service enforces access control to ensure only authorized users can perform actions. Interfaces are designed to support real time updates and consistent data flow between the client and backend.

4.2 Status Contribution — Joshua Dupati

For Status Meeting 1, Joshua contributed to the preparation of the RAD documentation by developing and organizing the Software Requirements Specification for the SP-5Red Grocery List App project. His work included defining the core functional requirements such as user account management and authentication, grocery list creation and editing, shared list collaboration and navigation flow. He also contributed to outlining system constraints, non-functional requirements, and analysis sections including use case and data flow. He ensured the document structure aligned with the project goal of creating a cross-platform mobile app that supports real-time grocery list synchronization between users.

5. Test Plan and Report

5.1 Test Plan

The test plan for the SP-5Red Grocery List App was designed to validate functionality across all major system components. The April 17, 2026 milestone included completion of both the Test Plan and Test Report.

5.2 Test Cases

TC #	Test Case	Expected Result	Status
TC-01	User Registration	User account created successfully with valid email/password	**Pass**
TC-02	User Login	User authenticated and granted access to grocery lists	**Pass**
TC-03	Create Grocery List	New grocery list created and visible on home screen	**Pass**
TC-04	Add Item to List	Item appears in list immediately after adding	**Pass**
TC-05	Update Item	Item updated and change reflected in list	**Pass**
TC-06	Delete Item	Item removed from list successfully	**Pass**
TC-07	Share List	Invited user can view and edit shared list	**Pass**
TC-08	Real-Time Sync	Changes reflect across all users simultaneously	**Pass**
TC-09	Unauthorized Access	Non-member blocked from accessing list	**Pass**
TC-10	Multi-User Editing	Concurrent edits handled without data loss	**Pass**

6. Version Control

6.1 Version Control Plan

GitHub will be used as the main version control system for this project. A shared repository will store all source code, documentation and configuration files. Each team member will work on their own branch when developing new features or fixing bugs.

The main branch will contain stable and tested code only. Changes will be merged into the main branch after review and basic testing. Commit messages will clearly describe the changes made. GitHub issues will be used to track tasks, bugs and progress throughout the semester.

6.2 Repository

GitHub Repository: github.com/asharulcoding/sp5-red-grocery-app

6.3 Risk Assessment

This project includes several risks that could impact development and delivery.

- Real-Time Synchronization Issues — issues with real time synchronization between users which could cause inconsistent list updates. This risk will be reduced by using reliable backend tools and testing synchronization early.
- Schedule Conflicts — conflicts between team members managed through weekly meetings, clear task assignments and regular check-ins.
- Integration Issues — issues when merging code from multiple contributors. Reduced by using feature branches, regular merges and basic testing before updates reach the main branch.

7. Conclusion / Summary

7.1 Project Summary

The SP-5Red Grocery List App is a cross-platform mobile application designed to help users create, manage, and share grocery lists in real time. The project successfully delivered a working prototype that supports user authentication, shared grocery lists, and real-time synchronization across devices.

The team followed Agile development practices throughout the semester, with regular weekly meetings, clear task delegation, and version control through GitHub. The project scope was met within the defined milestones from January through May 2026.

7.2 Deliverables Summary

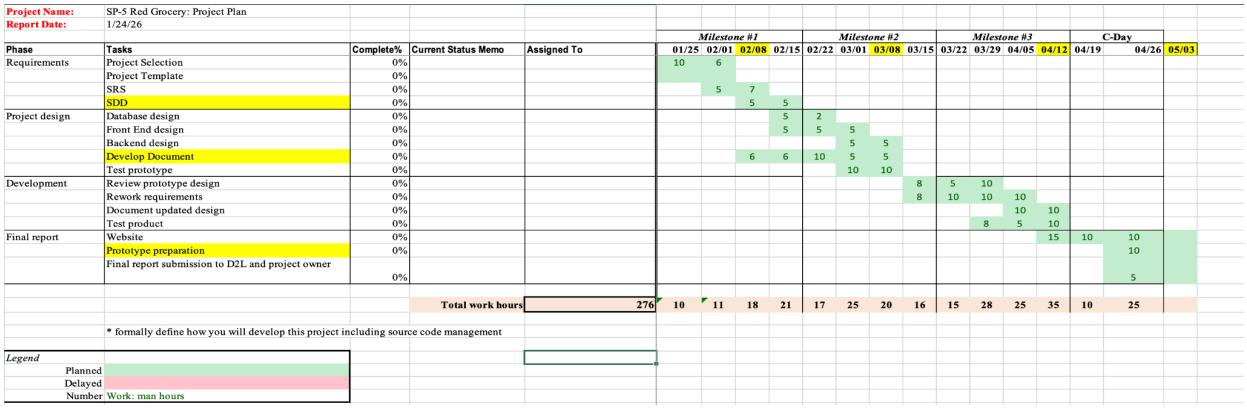
- Grocery List App available on multiple platforms supporting user accounts and shared grocery lists
- Database for user info and list data
- Individual and group mockups depicting information flow
- Real time list synchronization
- Prototype demonstration
- Test Data and Test Report
- Source Code

7.3 References

- CS 4850 Senior Project Guidelines
- Flutter. Flutter Documentation. Google. https://docs.flutter.dev/
- Firebase. Firebase Documentation. Google. https://firebase.google.com/docs
- Supabase. https://supabase.com/docs
- Dart. https://dart.dev/guides
- Balsamiq. https://balsamiq.com/docs/
- Canva. https://www.canva.com/help/

Appendix: Project Plan & Meeting Notes

A.1 Project Schedule (Gantt Chart)



Project Name:	SP-5 Red Grocery: Project Plan																							
Report Date:	1/24/26																							
					Milestone #1				Milestone #2				Milestone #3				C-Day							
Phase	Tasks	Complete%	Current Status Memo	Assigned To	01/25	02/01	02/08	02/15	02/22	03/01	03/08	03/15	03/22	03/29	04/05	04/12	04/19	04/26	05/03					
Requirements	Project Selection	0%			10	6																		
	Project Template	0%				5	7																	
	SRS	0%					5	5																
	SDD	0%																						
Project design	Database design	0%						5	2															
	Front End design	0%						5	5	5														
	Backend design	0%								5	5													
	Develop Document	0%					6	6	10	5	5													
	Test prototype	0%								10	10													
Development	Review prototype design	0%										8	5	10										
	Rework requirements	0%										8	10	10	10									
	Document updated design	0%													10	10								
	Test product	0%												8	5	10								
Final report	Website	0%														15	10	10						
	Prototype preparation	0%																10						
	Final report submission to D2L and project owner	0%																5						
			Total work hours	276	10	11	18	21	17	25	20	16	15	28	25	35	10	25						

* formally define how you will develop this project including source code management

Legend	
Planned	
Delayed	
Number	Work: man hours

A.2 Milestones

Date	Milestone
Jan 25, 2026	Project plan defined; work estimate created; required tools and frameworks chosen
Feb 8, 2026	Functional requirements defined; wireframe mockups; database design; SRS and SDD complete
Mar 8, 2026	Development documentation; Phase 1 capabilities (user creation, grocery list, member invitation)
Mar 17, 2026	Phase 1 complete; working prototype; presentation; begin Phase 2 (monetization, 3rd party data)
Apr 17, 2026	Test Plan and Test Report
May 3, 2026	Final report package; all app components complete; presentation video; source code

A.3 Meeting Notes

January 31, 2026 — 11:00 AM — Google Meet (11 min)

Attendees: Joshua Dupati, Jacob Brockel
Agenda: Review and analyze completed SRS and SDD sections; delegate remaining items

Notes:

- Discussion between group members occurs daily via iOS group chat
- Meeting times discussed to be moved prior or after class times on Tuesdays or Thursdays
- Action items are being completed ahead of schedule

Action Items:
- Section 5, 6, and 7 of SDD to be completed by Ash
- Review and finish formatting for SRS and SDD

Next Meeting: *Ensure SRS and SDD are complete prior to February 8th*

February 3, 2026 — 1:45–2:15 PM — In Person (30 min)

Attendees: Joshua Dupati, Jacob Brockel, Ash Arul
Agenda: Review deliverables

Notes:
- Reviewed deliverables
- Talked about the app and agreed with the requirements/scope

Action Items:
- Look into Flutter
- More research

Next Meeting: *Plan the Development phase and start working on the project*

February 10, 2026 — 1:45–2:15 PM — In Person (10 min)

Attendees: Joshua Dupati, Jacob Brockel, Ash Arul
Agenda: Plan Development

Notes:
- Josh will work on the front end
- Ash will create mockups

Action Items:
- Look into Flutter
- More research
- Plan
- Code

Next Meeting: *Check up on the work*

March 3, 2026 — 1:45–2:15 PM — In Person (10 min)

Attendees: Joshua Dupati, Ash Arul
Agenda: Documentation Development

Notes:
- Josh will continue to work on the front end
- Jacob will work on the back end
- Everyone will work on the development doc due

Action Items:
- Work

Next Meeting: *Get ready for the presentation*

March 17, 2026 — 1:45–2:15 PM — In Person (10 min)

Attendees: Joshua Dupati, Ash Arul, Jacob Brockel
Agenda: Presentation
Notes:

- Josh added his parts to the presentation of the UI
- Jacob added his parts of the presentation
- Practice

Action Items:

- Edit the presentation

Next Meeting: *Finishing up the project — Website, App, and turning in the whole portfolio*

March 31, 2026 — 2:00–3:00 PM — In Person (1 hr)

Attendees: Joshua Dupati, Ash Arul, Jacob Brockel
Agenda: Presentation Practice
Notes:

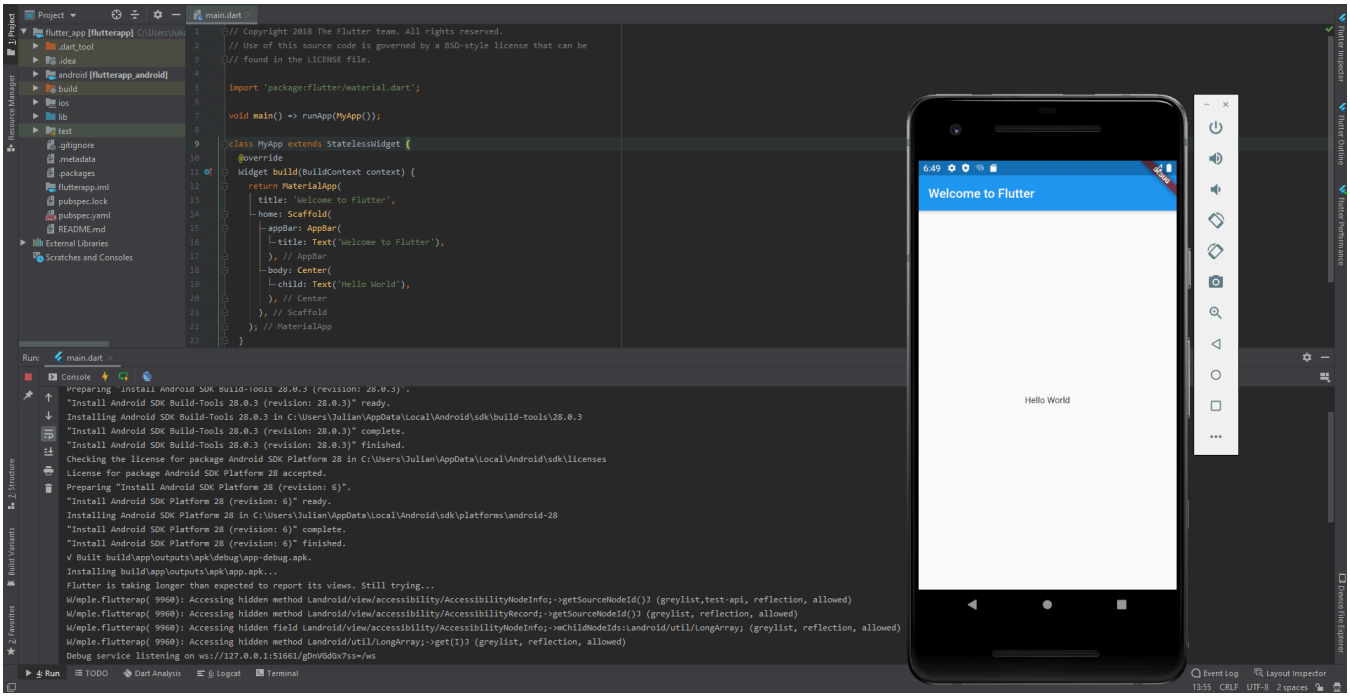
- Finalized the presentation
- Assigned speaker roles
- Practiced it

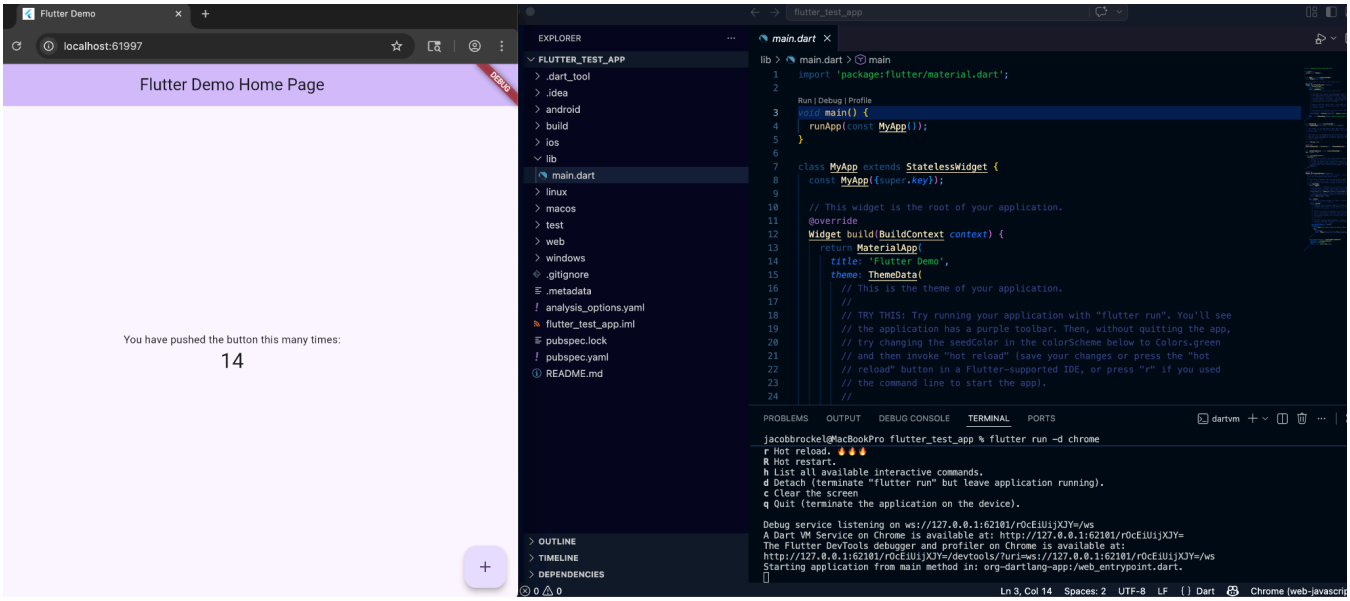
Action Items:

- Practiced it

Next Meeting: *Sync — Test Cases, Website*

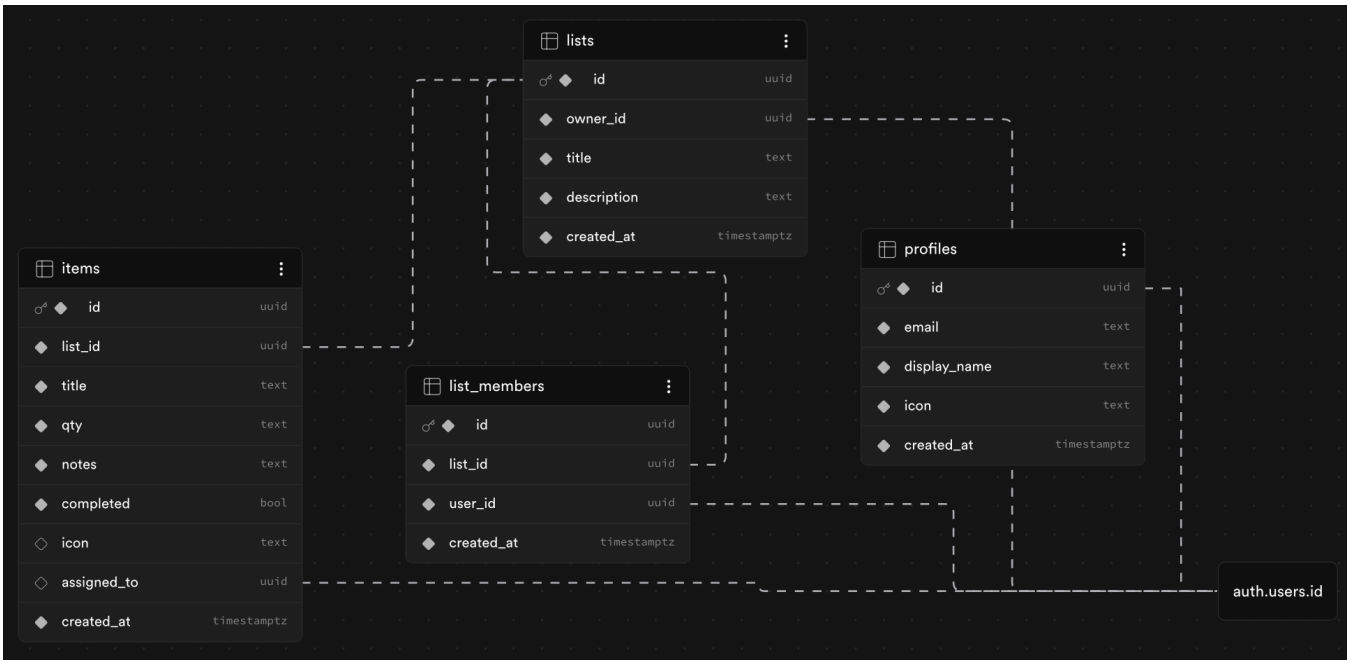
A.4 Flutter Dart Tutorial



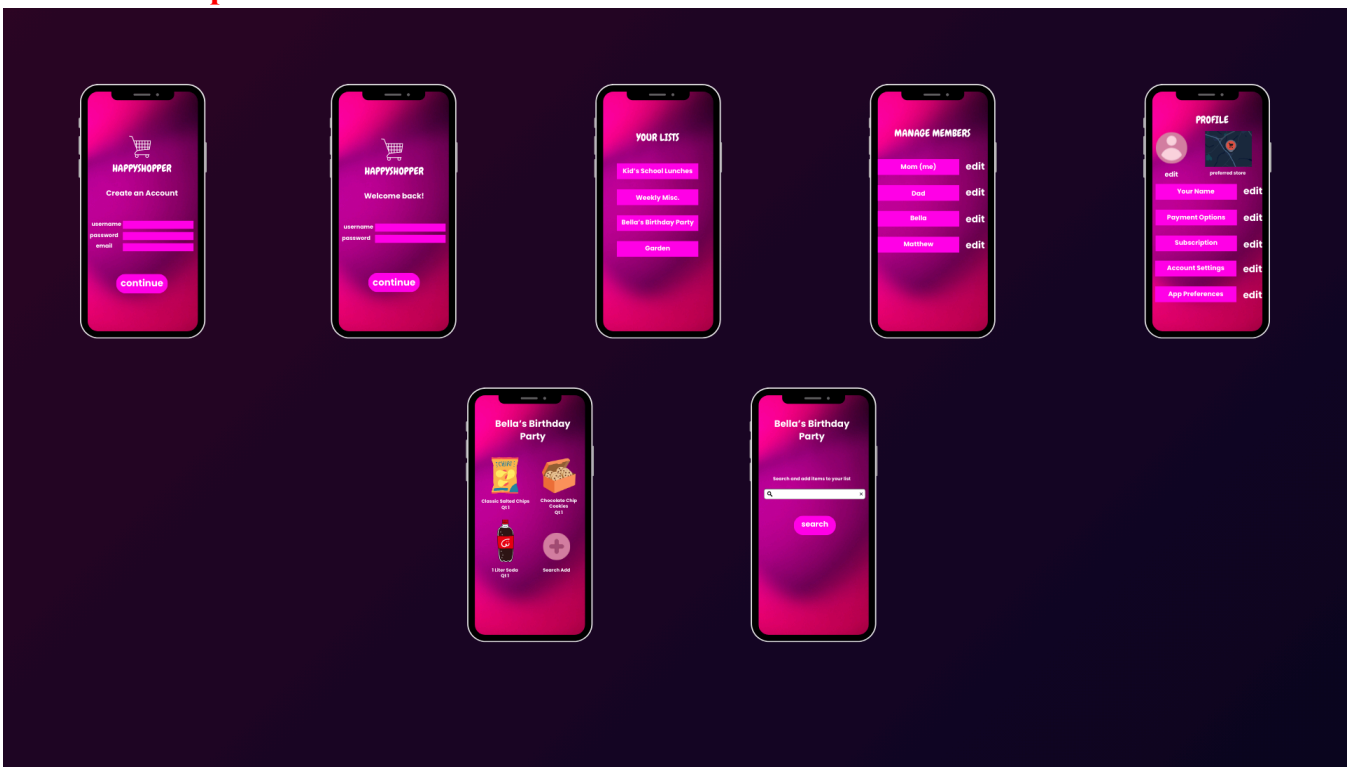


Proof of Flutter demo running after both the Flutter and Dart tutorials were completed

A.5 Database Diagram



A.6 UI Mockup


HAPPYSHOPPER
Create an Account
username
password
email
continue
HAPPYSHOPPER
Welcome back!
username
password
continue
YOUR LISTS
Kid's School Lunches
Weekly Misc.
Bella's Birthday Party
Garden
MANAGE MEMBERS
Mom (me)
edit
Dad
edit
Bella
edit
Matthew
edit
PROFILE
edit
preferred store
Your Name
edit
Payment Options
edit
Subscription
edit
Account Settings
edit
App Preferences
edit
Bella's Birthday Party
Classic Salted Chips
Q:1
Chocolate Chip Cookie
Q:1
1 Liter Soda
Q:1
Search Add
Bella's Birthday Party
Search and add items to your list
search